



哈爾濱工業大學(深圳)

HARBIN INSTITUTE OF TECHNOLOGY, SHENZHEN

Kairix

基于Rust的支持RISC-V 64和Loongarch64 双架构操作系统

队员：颜晨、雷鑫言、萧鹏 指导老师：夏文、仇洁婷

2026.6.30 初赛结束，排名和得分详情如下，总分2550，排名第七：

2	T202610574999863	夺金入场	华南 师范 大学	28	2026- 06-24 22:29:31	102.0	102.0	102.0	102.0	57.0	57.0
3	T2026100069910965	起名废	北京 航空 航天 大学	60	2026- 06-20 21:31:17	102.0	102.0	102.0	102.0	54.0	54.0
4	T202610617999569	每天睡满八小时	重庆 邮电 大学	200	2026- 06-24 02:43:21	102.0	102.0	102.0	102.0	54.0	54.0
5	T202610459999616	PulseOS	郑州 大学	155	2026- 06-24 10:25:57	102.0	102.0	102.0	102.0	54.0	54.0
6	T202610112999587	MyGO!!!!	太原 理工 大学	39	2026- 06-22 16:35:42	100.0	102.0	100.0	102.0	54.0	53.0
7	T2026181239910993	Unicus	哈尔 滨工 业大 学 (深 圳)	42	2026- 06-24 13:27:34	102.0	102.0	102.0	102.0	54.0	54.0

总分: 2550.3512467086457

得分详情

测试点	glibc-la	glibc-rv	musl-la	musl-rv		总分
basic	102	102	102	102		408
busybox	54	54	54	54		216
cyclictest	4.005370569280344	4.0	0.0	4.0		12.005370569280345
iozone	30.282548889212105	30.86234527825802	28.530791257089803	27.686189341363455		117.36187476592337
iperf	6.0	6.0	6.0	6.0		24.0
libcbench	31.682885418296983	28.688114893798797	29.110494634970266	27.18339106633986		116.6648860134059
libctest	-	-	216	217		433
lmbench	39.474005789478035	40.77088989140956	41.51669368003636	40.92608347827112		162.68767283919507
ltp	5229	5280	5232	5286		21027
lua	9	9	9	9		36
netperf	5.905927080685355	6.4631932730039034	6.1645638562095195	6.097758310942241		24.63144252084102
总分	5511.350737746953	5561.78454333647	5724.322543428306	5779.893422196917		2550.3512467086457

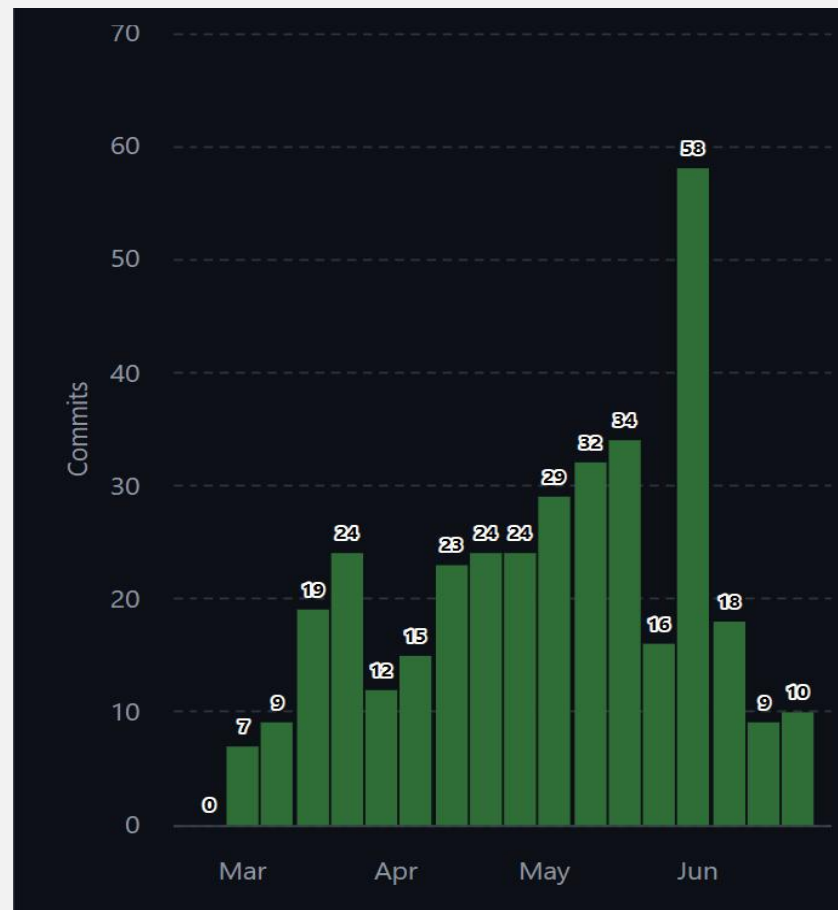


Kairix内核工作量:

- 搭建时间: 约4个月
- 内核 rust 代码量: **4.8w+** 行
- commits 数量: **380+**

```
183 text files.
183 unique files.
0 files ignored.

github.com/AlDanial/cloc v 1.98 T=0.09 s (2006.9 files/s, 641896.5 lines/s)
-----
Language          files      blank      comment      code
-----
Rust                176        4921        4187        48753
Assembly            4           60          50          376
Linker Script       3           21          12          152
-----
SUM:                183        5002        4249        49281
-----
```



Kairix项目简介

成员分工：

- 颜晨：文件系统、异常机制、进程调度
- 雷鑫言：内存管理、内存多架构实现
- 萧鹏：信号、进程间通信、网络

模块	完成情况
进程管理	有栈协程、多核调度。进程线程分离设计。
内存管理	内核空间动态映射、缺页异常驱动的懒分配、写时复制、支持零页分配、用户指针检查、动态链接、共享内存映射。
文件系统	类 Linux 的虚拟文件系统、路径解析缓存、页缓存加速、延迟写、支持挂载、支持 Ext4/Ext3/Ext2/Fat32磁盘文件系统，内存文件系统、进程文件系统、设备文件系统。
信号机制	支持标准信号与实时信号的相关接口、支持用户自定义信号处理、线程信号掩码、同步异常信号投递
设备驱动	支持硬件中断、MMIO驱动、PCI驱动和部分串口驱动、解析部分设备树信息
网络模块	自研网络栈、支持TCP UDP套接字、支持本地回环设备、支持IPv4协议栈
架构管理	基于开源多架构硬件抽象层polyhal、支持RISC-V、LoongArch双架构



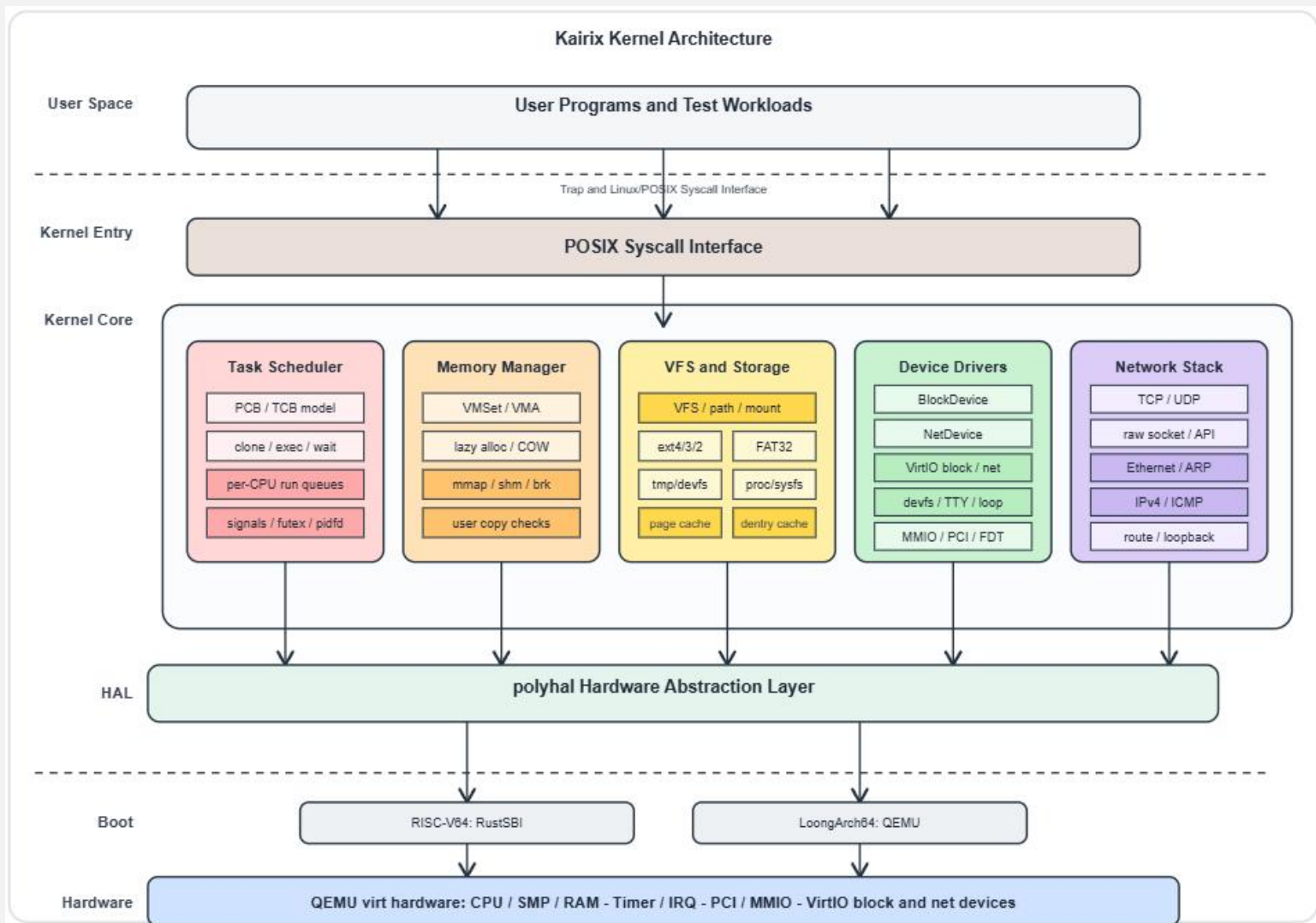
Kairix内核整体架构

```
Kairix/
├── os/                                # 内核主体代码：进程、内存、VFS、网络、驱动、系统调用等
│   └── src/
│       ├── main.rs                  # 内核入口，完成初始化后进入任务调度
│       ├── config.rs
│       ├── logging.rs              # 日志初始化
│       ├── console.rs              # 内核控制台输出封装
│       ├── error.rs                # 内核错误码与系统调用错误类型
│       ├── lang_items.rs           # no_std panic/语言项支持
│       ├── timer.rs                # 时钟与定时器相关逻辑
│       ├── embedded.rs             # 启动时内置文件安装
│       ├── sbi.rs                  # RISC-V SBI 调用封装
│       ├── sbi_la.rs               # LoongArch 固件调用封装
│       ├── entry.asm               # 早期汇编入口
│       ├── link_app.S              # 用户程序链接辅助
│       ├── linker-*.ld             # 各架构链接脚本
│       └── arch/                   # 架构相关代码，RV启动代码
│           ├── boards/             # 板级配置，当前主要面向 QEMU virt
│           ├── devices/            # 通用设备抽象
│           ├── drivers/            # 设备驱动，包含 VirtIO 块设备与 PCI 探测
│           └── fs/                 # 文件系统子系统
│               ├── vfs/             # VFS 抽象
│               ├── devfs/           # 设备文件系统
│               ├── procfs/          # proc 文件系统
│               ├── sysfs/           # sys 文件系统兼容节点
│               ├── tmpfs/           # 内存文件系统
│               ├── fat32/           # FAT32 文件系统适配
│               ├── lwext4/          # ext4 文件系统适配
│               ├── page/            # 页缓存
│               └── etc/             # /etc 下的内置配置文件
│           ├── mm/                 # 内存管理
│           ├── net/                 # 网络协议栈
│           ├── socket/             # socket 层
│           ├── sync/               # 同步原语
│           ├── syscall/            # 系统调用实现
│           ├── task/               # 进程/线程管理、调度器、PID、上下文切换
│           ├── trap/               # trap/异常/中断处理入口与上下文
│           └── vendor/             # 内核构建使用的离线依赖
```

```
├── user/                             # 用户态运行时库与测试/示例程序
│   ├── src/
│       ├── lib.rs                  # 用户态运行时入口、堆初始化、系统调用安全封装
│       ├── syscall.rs              # 用户态系统调用号与 ecall/syscall 汇编封装
│       ├── console.rs              # 用户态 print/println 与输入输出辅助
│       ├── lang_items.rs           # 用户态 no_std panic/语言项支持
│       ├── linker.ld               # 用户程序链接脚本
│       └── bin/                    # 用户程序与测试入口
│           ├── initproc.rs         # 第一个用户进程
│           ├── user_shell.rs       # 简易 shell
│           ├── ls.rs               # 目录查看工具
│           ├── ping.rs / ping2.rs  # 网络连通性测试
│           ├── tcp_test.rs / tcp_socket_test.rs # TCP/socket 测试
│           ├── usertests*.rs       # 用户态综合测试
│           ├── libctests_*.rs      # libc-test 启动入口
│           ├── libcbench.rs        # libc-bench 启动入口
│           └── hello_world.rs 等    # 基础示例与回归测试程序
│   └── vendor/                     # 用户程序构建使用的离线依赖
```




Kairix内核整体架构

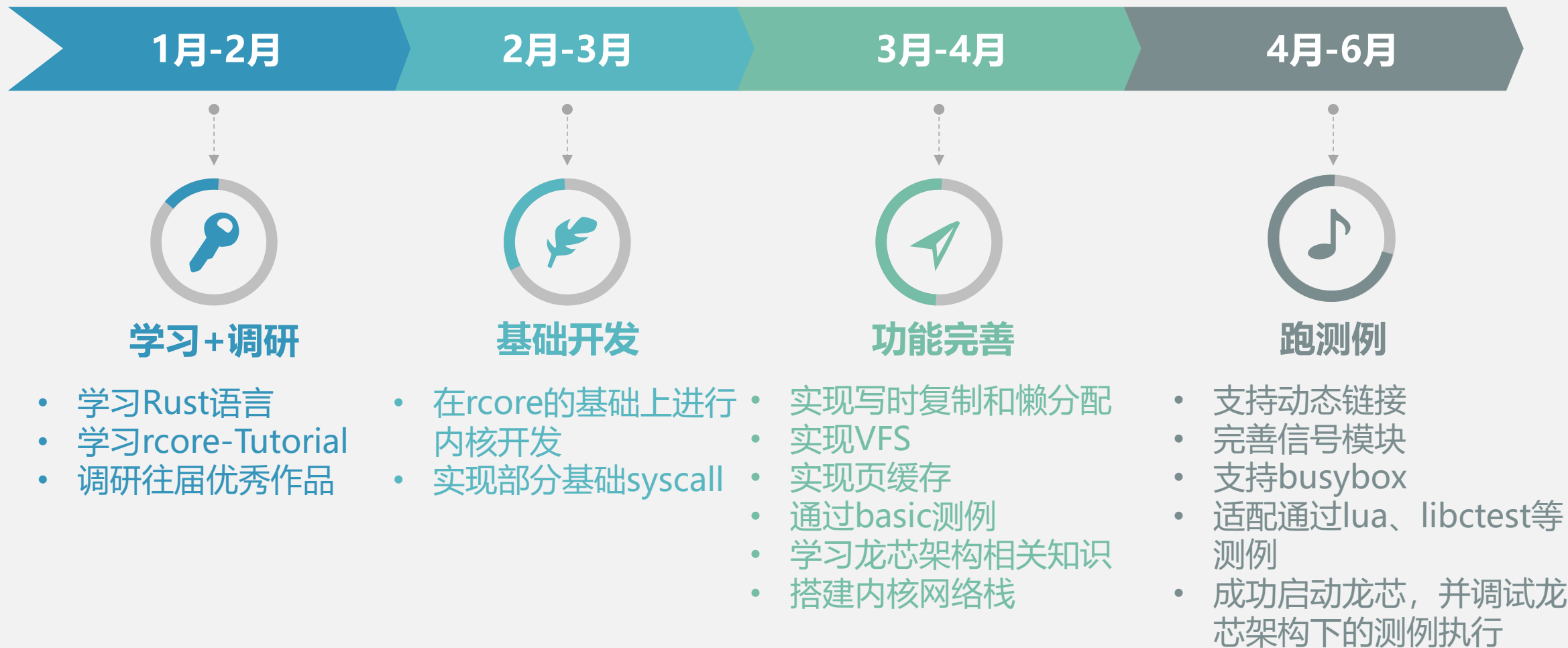




Kairix内核开发历程



Kairix开发历程





Kairix核心模块内容介绍



1. 多架构设计

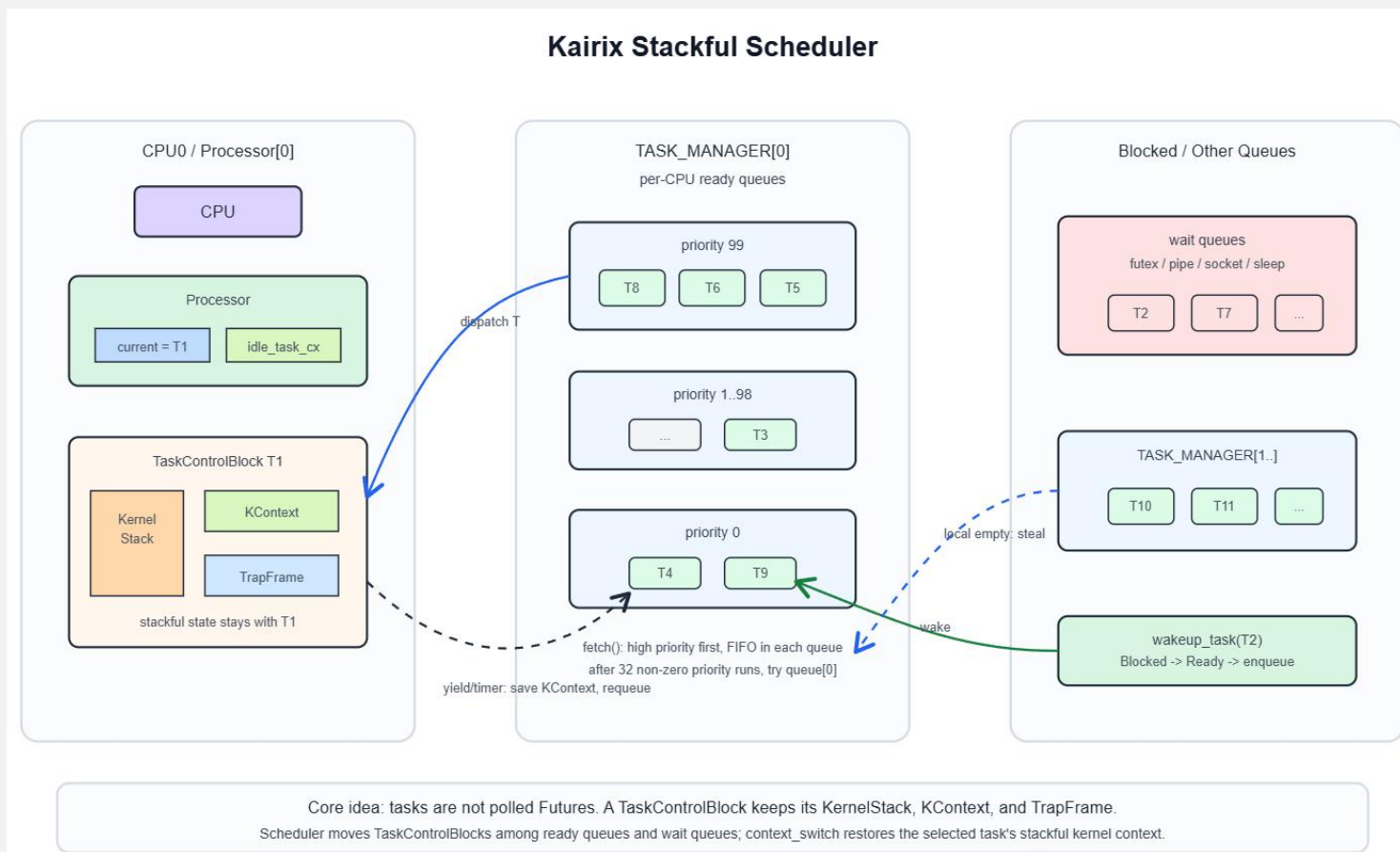
Kairix内核目前通过移植和修改polyhal硬件抽象层实现了对RISC-V和LoongArch两种指令集架构的支持。

具体而言，Kairix利用Rust语言的条件编译机制，利用polyhal第三方库，实现硬件抽象层屏蔽了不同架构的底层差异。其核心理念是通过条件编译实现对应架构代码的自动选择，从而将不同架构硬件的功能抽象出来，供内核统一使用。

2. 进程调度

为了确保任务调度之间的公平性和优先级，Kairix采用的方案是：

- ①同一优先级队列中采用FIFO调度
- ②不同优先级之间默认从高优先级到低优先级扫描。
- ③不同优先级之间，当连续调度非零优先级任务达到一定次数后，调度器会优先尝试从最低优先级队列中取出一个任务运行；若该队列为空，则重新从高到低扫描所有优先级队列。





3. 内存管理

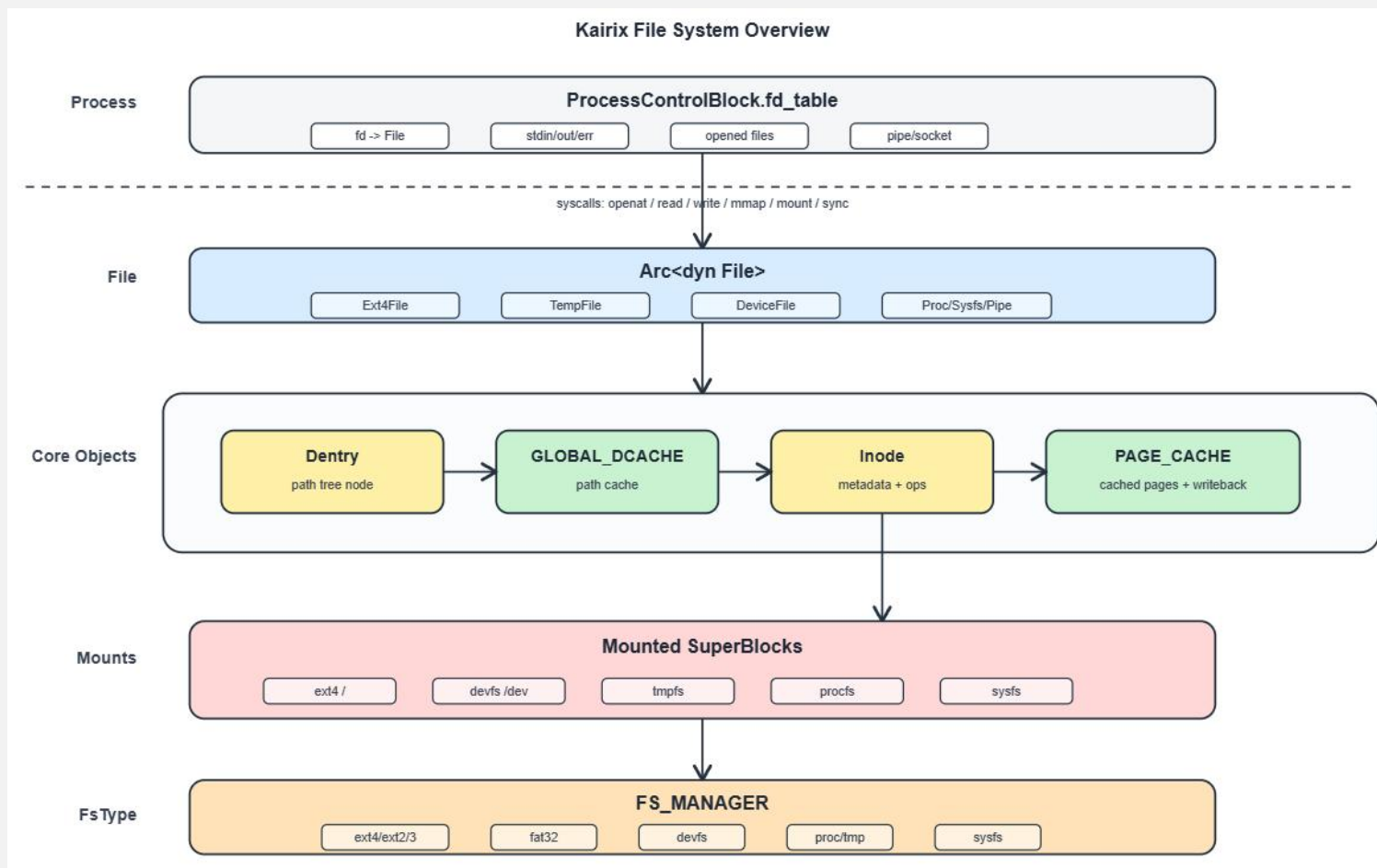
- ① Kairix 支持多区间分配并优先复用释放页；同时采用伙伴算法为内核动态结构分配内存
- ② 为降低上下文切换开销，Kairix 采用内核与用户空间共享同一套高地址内核映射的设计，即所有进程页表均复制内核页表项，陷入内核后无需切换页表即可直接访问内核代码、数据及物理内存映射区。
- ③ 用户内存空间采用懒分配、写时复制，同时支持匿名、文件映射及共享内存，通过 VMA 记录文件偏移、权限等元数据，实现按需加载与私有复制。



4. 文件系统

Kairix的文件系统以Linux文件系统为参考，充分结合了Rust的特性进行多态抽象，对VFS进行了设计和优化。

目前Kairix支持多文件系统，主要支持ext4，并且基本支持Fat32。此外，Kairix还设计了很多非磁盘文件系统，包括但不限于procfs、tmpfs、devfs，设计理念符合“万物皆文件”的思想。





4. 文件系统

Kairix在VFS层实现了统一的页缓存机制，使文件读写尽可能在内存中完成，避免重复访问块设备。同时，mmap文件映射也可以通过页缓存获得文件页对应的物理帧，从而将文件内容映射到用户地址空间中。

我们采用全局页缓存，回收策略采用LRU和延迟写回，虽然牺牲了一部分的生命周期管理的直观性，但换来了统一的管理和回收。

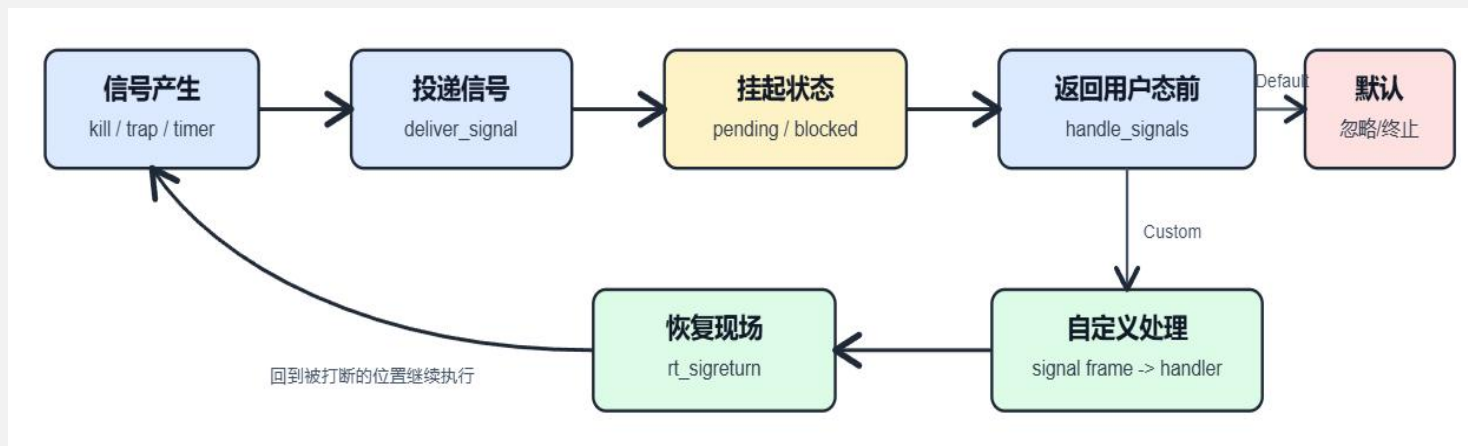


5. 进程间通信

5.1 信号机制

信号是一种轻量级的异步通知机制。Kairix中的信号既可以由一个进程发送给另一个进程，也可以由内核在异常、定时器、管道错误等场景下主动投递给进程。

我们的信号处理机制主要是参考的Linux的语义，支持1到64号信号，并为每个信号维护默认行为。信号的处理方式包括三种：默认处理、忽略处理和用户自定义处理函数。其中SIGKILL和SIGSTOP不能被捕获或忽略，用于保证内核始终可以强制终止或停止目标进程。





5. 进程间通信

5.2 管道

管道 (Pipe) 是一种典型的进程间通信机制，用于在进程之间传递字节流数据。Kairix中的pipe底层由共享的PipeRingBuffer实现，读端和写端分别是两个Pipe文件对象，它们共同引用同一个环形缓冲区。

5.3 共享内存

共享内存是一种高效的进程间通信机制。与管道这类字节流通信不同，共享内存允许多个进程将同一组物理页映射到各自的虚拟地址空间中，从而直接通过内存读写交换数据，避免了频繁的数据拷贝。

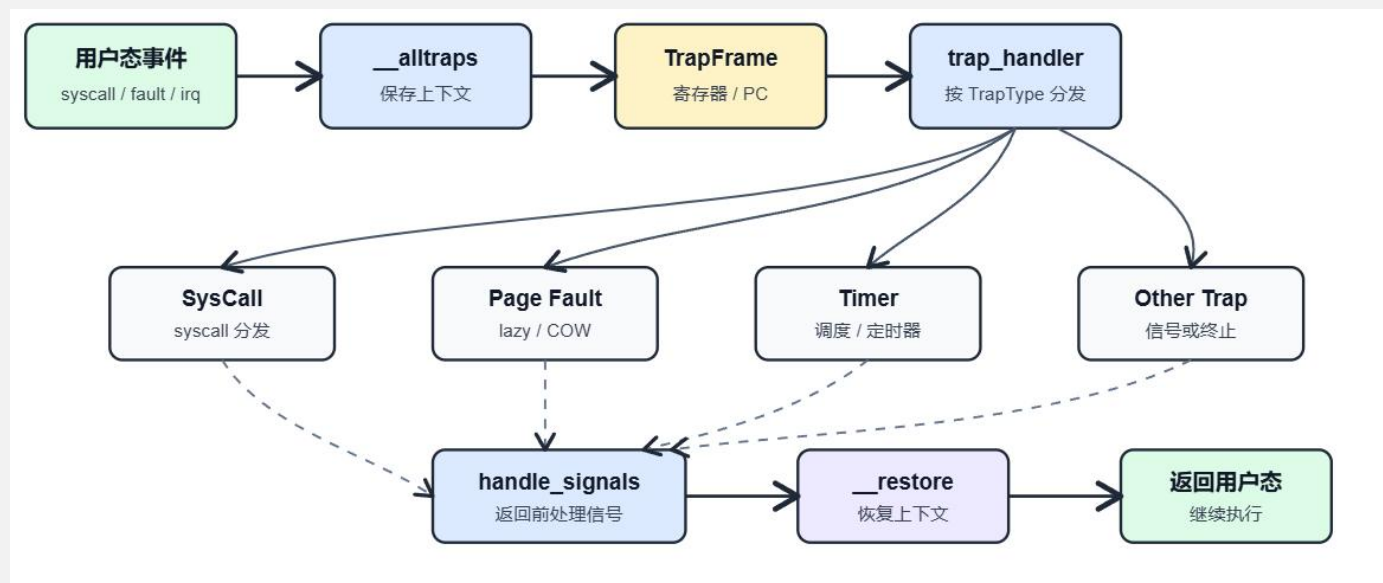
Kairix实现了完整的System V共享内存机制，允许多个进程共享同一块物理内存区域，实现高效的数据交换。



6. 系统调用

Kairix提供Linux风格的系统调用接口，使用户程序可以通过统一方式请求文件系统、进程管理、内存管理、网络、信号等内核功能。

在trap这方面，我们团队复用了开源库，做出了优化，Kairix通过polyhal和polyhal-trap抽象不同架构的trap细节，使RISC-V64和LoongArch64能够复用大部分trap处理逻辑。



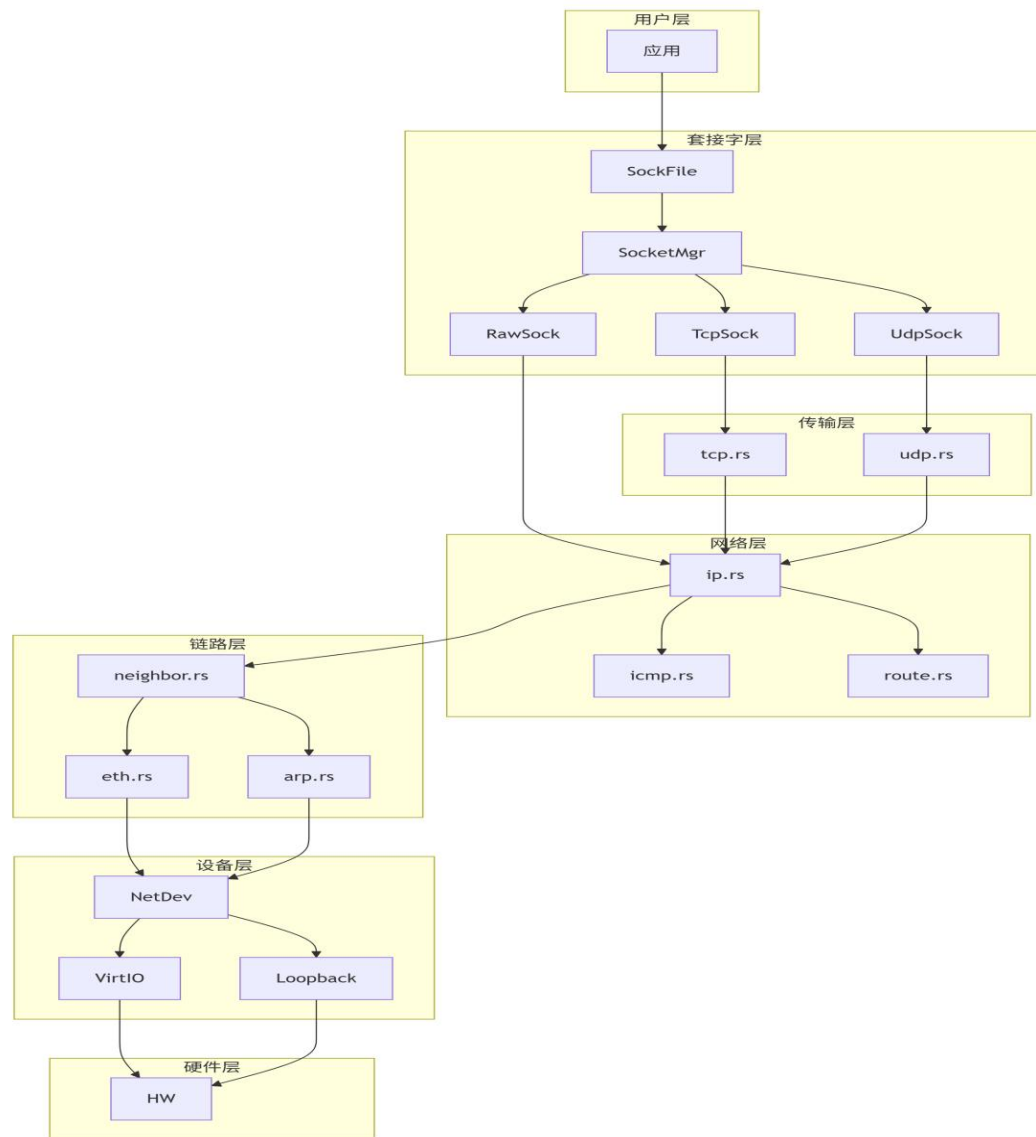
6. 系统调用

类别	代表系统调用	功能描述
进程与线程管理	clone, clone3, execve, exit	进程和线程创建、执行、退出、等待
内存管理	brk, mmap, mprotect, msync	虚拟内存映射、堆管理、权限和同步
文件系统	openat, close, read, write	文件打开、读写、状态查询
目录与路径	mkdirat, chdir, unlinkat	目录、路径和链接管理
文件系统挂载	mount, fsync, syncfs	文件系统挂载、卸载和同步
网络通信	accept, connect, sendto	socket创建、连接和数据传输
信号处理	rt_sigaction, kill	信号设置、屏蔽、等待、发送和返回
时间管理	clock_gettime, setitimer	时间获取、休眠和定时器
同步与IPC	futex, shmget, shmat	futex、共享内存和事件通知
I/O控制	fcntl, ppoll, pselect6	设备控制和I/O多路复用
用户与权限	getuid, geteuid, getgid	用户身份和权限管理
系统信息	sysinfo, syslog	系统信息和资源限制
兼容接口	inotify, fanotify	面向Linux测试的兼容接口



7. 网络模块

Kairix的网络模块是一个完全自主实现的轻量级TCP/IP协议栈，支持IPv4、ARP、ICMP、UDP和TCP等核心协议，并提供了与POSIX兼容的套接字接口。网络模块采用清晰的分层架构，自底向上包括设备层、链路层、网络层、传输层和套接字层，各层之间通过统一的数据包缓冲区和trait接口进行交互，降低了模块间的耦合度，便于后续的扩展和维护。





8. 设备驱动

Kairix当前主要面向QEMU virt平台，支持RISC-V64和LoongArch64两种架构。驱动层主要负责块设备、字符设备和网络设备的初始化与访问，并通过统一抽象向上层文件系统、网络协议栈和用户程序提供服务。

设备驱动的实现以VirtIO设备为核心：块设备主要服务于ext4根文件系统，字符设备通过devfs暴露给用户程序，网络设备则接入Kairix自研网络协议栈。



哈爾濱工業大學(深圳)
HARBIN INSTITUTE OF TECHNOLOGY, SHENZHEN

Kairix

设计文档

参赛队名 Unicus

队伍成员 颜晨、雷鑫言、萧鹏

指导老师 夏文、仇洁婷

2026 年 6 月 30 日



比赛总结



Kairix内核当前成果:

- ① 测例通过方面: 我们通过了除部分LTP测试之外的所有测试点, 在排行榜上位于前列
- ② 文件系统与存储管理: 接入并优化了lwext4, 重构VFS以支持多种文件系统类型, 引入页缓存机制显著提升读写性能, 并完善了相关系统调用。
- ③ 进程与信号机制: 实现了标准进程管理与Linux兼容的信号处理机制, 支持动态链接提升加载效率, 并通过futex和共享内存增强了进程间同步与通信能力。
- ④ 内存管理: 构建了支持分页、虚拟内存、缺页按需分配和写时复制的内存管理系统, 通过VMA与页表权限协同管理用户地址空间, 并支持mmap、brk等多种机制。



Kairix内核当前成果:

- ⑤ 调度与多核支持: 具备基础的多核启动与调度能力, 支持任务并发执行, 为后续负载均衡和调度优化提供了基础。
- ⑥ 网络支持: 自研网络协议栈支持VirtIO-net、ARP、IPv4、ICMP、TCP/UDP及raw socket, 可在QEMU环境中进行ping、TCP/UDP连接及iperf等网络测试。
- ⑦ 硬件抽象与平台适配: 通过polyhal屏蔽架构差异, 同一份代码可在不同硬件平台上运行, 具备良好的可移植性和可维护性。



未来的发展计划：

- ① 真实物理板的适配：支持 RISC V 以及龙芯开发板
- ② 网络功能的完善：支持更多的功能，比如http、https和ssh
- ③ 优化文件系统：继续优化页缓存，同时补充负缓存等优化策略
- ④ 图形化界面：提升用户交互体验
- ⑤ 锁的完善：当前锁的种类还是不够，读写锁用的还是库里面的
- ⑥ 运行现实生活的应用：例如数据库



关于AI工具的使用：

大模型本质上是基于概率建模和大规模数据训练得到的工具。因此，在操作系统开发这种对正确性要求极高的工程中，不能把核心判断完全交给AI。所以在开发的过程中，我们团队利用了AI加快进度的开发，但是有限度的利用AI工具，我们认为使用AI的目的是：**加快迭代的速度，减少重复工作量，而不是抛弃大脑。**

我们使用的模型是kimi2.6+gpt5.5+deepseek。AI工具确实显著提高了我们的开发效率，但我们始终坚持有限度地使用AI：**核心架构由人来设计，关键修改由人来审查，最终正确性由测试结果和代码阅读共同确认。**



AI工具的主要使用场景：

- ①快速学习：让AI通读内核源码，快速指出关键结构体和调用路径，降低大型代码库的理解成本。
- ②代码生成：AI擅长生成模式固定、重复性强的代码，如/dev/zero、/dev/urandom等结构相似的文件对象，可减少机械重复工作，但具体语义逻辑仍需人工把关。
- ③辅助调试：通过日志定位问题后让AI初步诊断并给出修改建议，但AI上下文有限且易产生幻觉，最终判断和确认仍需人为介入。
- ④生成测试：AI可快速生成覆盖边界输入的测试程序，但测试有效性和Linux语义符合性需人工审查。
- ⑤文档润色：先人工撰写初版，再让AI润色并选择性采纳，避免直接生成导致内容缺乏深度和思考感。



参考的优秀内核:

- TitanixOS: no_std 锁
- rCore: 用户态程序
- Polyhal、DelOn1x: 多架构设计
- PhoenixOS、Chronix、NighthawkOS: 设计文档
- Chronix: 文件系统架构思路



哈爾濱工業大學(深圳)

HARBIN INSTITUTE OF TECHNOLOGY, SHENZHEN

感谢观看

队员：颜晨、雷鑫言、萧鹏 指导老师：夏文、仇洁婷